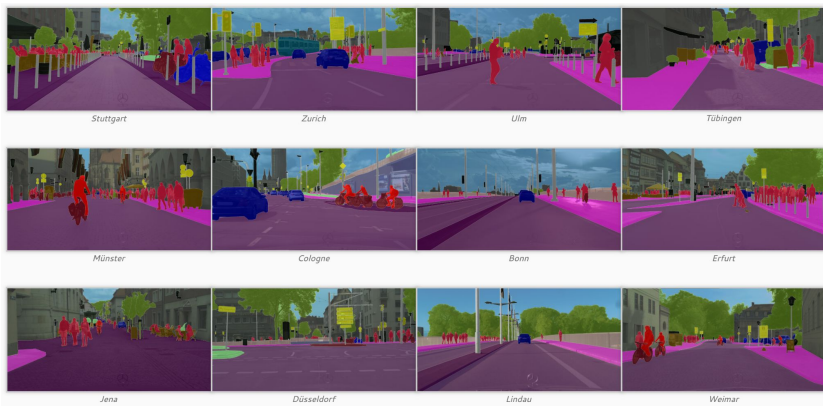


Segmentacja obrazów dla pojazdów autonomicznych

przy użyciu algorytmu k-means na przykładzie cityscapes-dataset



<https://www.cityscapes-dataset.com/examples/>



Opis projektu

Segmentacja obrazów dla pojazdów autonomicznych

Cel projektu:

Segmentacja obrazów przy użyciu algorytmu *k-means*.

W projekcie należy:

- rozszerzyć opis piksela o współrzędne (x, y) ,
- porównać segmentację:
 - tylko na podstawie koloru,
 - z uwzględnieniem położenia pikseli,
- przeanalizować wpływ liczby klastrów,
- ocenić jakość segmentacji przy użyciu IoU,
- przygotować wizualizacje segmentacji.

Przykładowe dane:

- Cityscapes Dataset.

Problem

Pojazdy autonomiczne muszą w czasie rzeczywistym rozumieć otoczenie, identyfikować granice obiektów (przeszkód, ludzi, innych pojazdów) na obrazach z kamer.



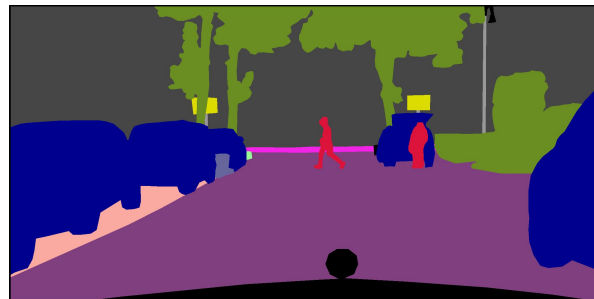
Przykładowe dane

Cityscapes dataset - zbiór zdjęć z kamer zamontowanych w samochodach jeżdżących po niemieckich miastach. W naszym przypadku **150 zdjęć z Kolonii**

Image



GroundTruth

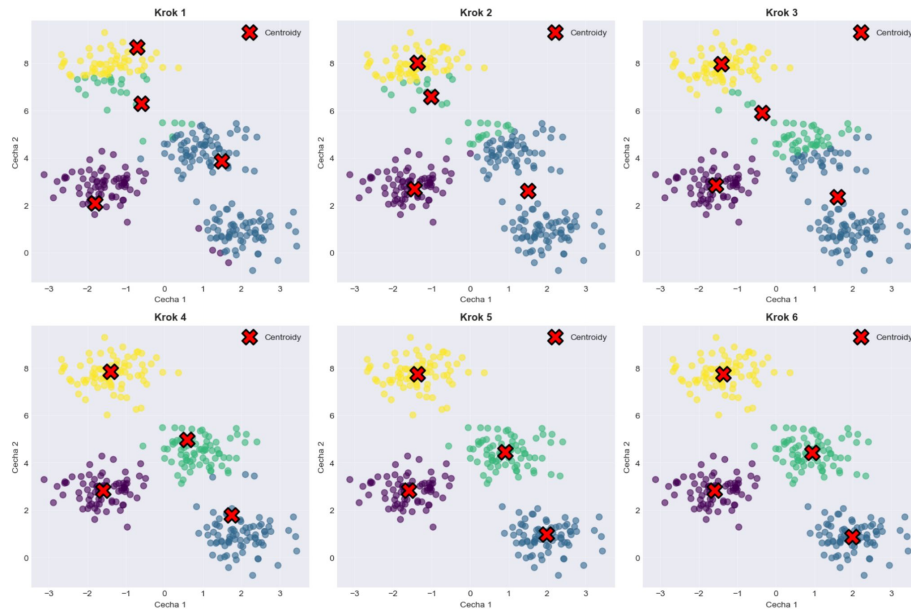


Cityscapes ground-truth classes

road	sky
sidewalk	person
building	rider
wall	car
fence	truck
pole	bus
traffic light	train
traffic sign	motorcycle
vegetation	bicycle
terrain	

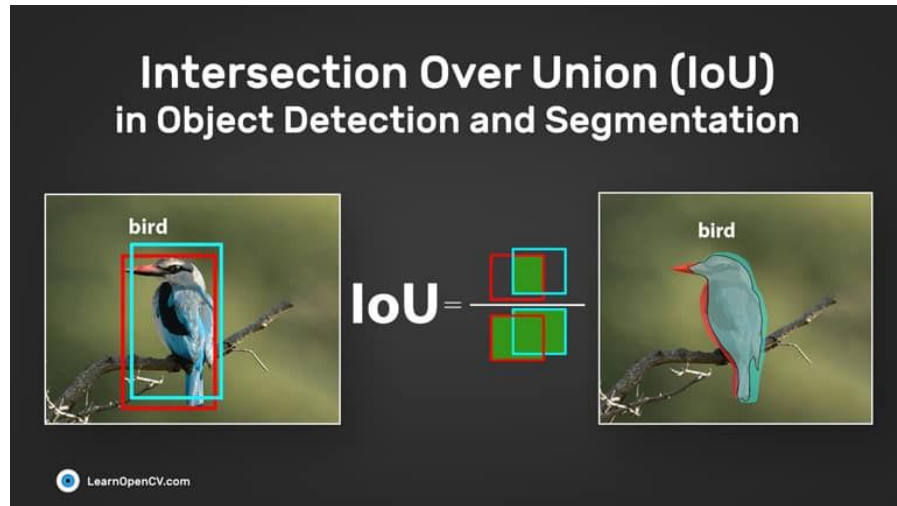
Algorytm k-means

Algorytm **K-means** automatycznie porządkuje chaotyczne dane, dzieląc je na zadaną z góry liczbę (**k**) spójnych grup. Jego głównym zadaniem jest odnalezienie naturalnych podobieństw w taki sposób, aby elementy trafiające do jednego klastra były do siebie jak najbardziej zbliżone cechami, a poszczególne grupy wyraźnie się od siebie różniły.



Metodologia

Opis piksela oprócz wektora RGB rozszerzyliśmy o jego współrzędne (x, y). Testowaliśmy algorytm w oparciu o parametr k (liczba klastrów) oraz λ (waga współrzędnych względem koloru). Dla każdego wyniku liczyliśmy IoU, porównując wynik z GroundTruth.



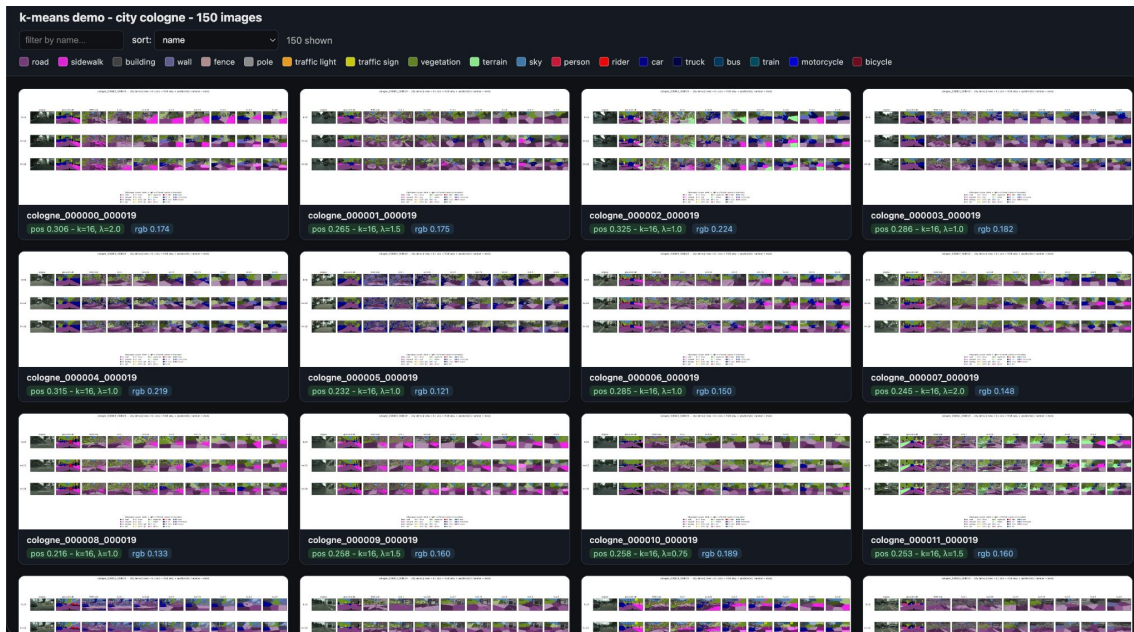
Metodologia

Kolorowanie w plikach zostało zmienione na takie które odpowiada kolorom z GroundTruth. Koloruje kolorami z GT z datasetu i one odpowiadają poszczególnym obiektom.

1. kmeans robi klastry
2. dla każdego klastra sprawdza dla każdego pixela jakiemu obszarowi z GT odpowiada
3. klastrowi przypisuje kolor tego obszaru z GT, któremu najwięcej pixeli odpowiada, ale żeby wizualnie się nie zlewały klastry w jedno, to robi raz jaśniej raz ciemniej

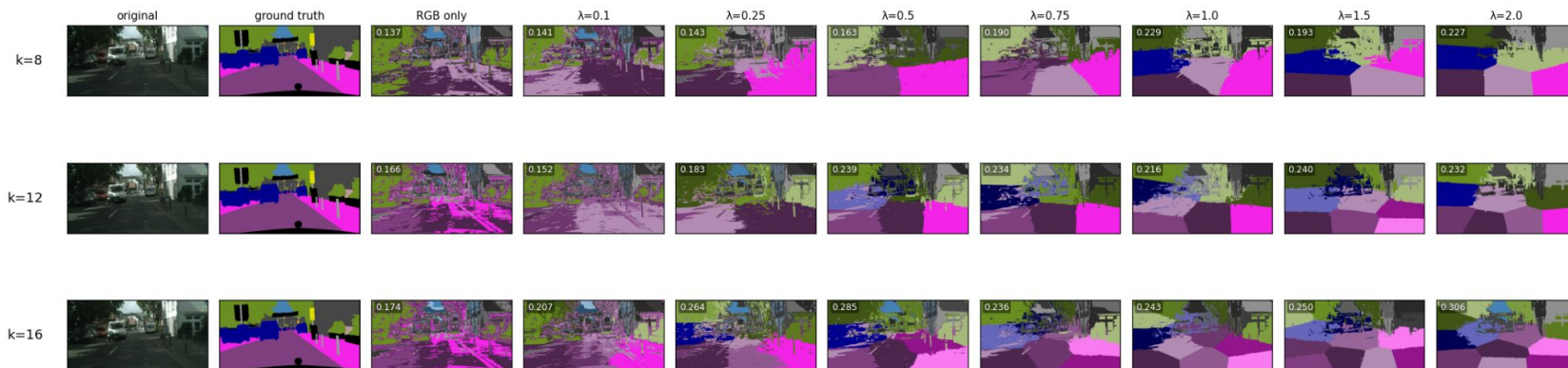
Metodologia

Do sprawnego przeglądania
wyników algorytmu
zrobiliśmy plik HTML w
którym można wygodnie
sortować po IoU.



Metodologia

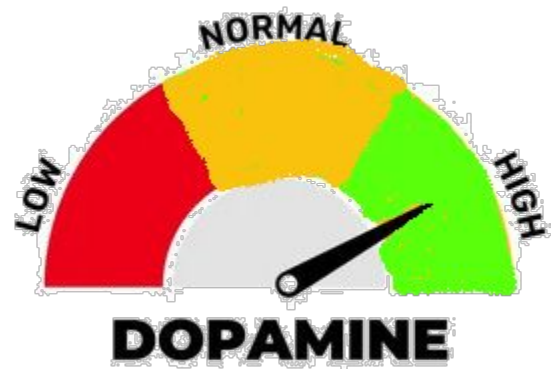
cologne_000000_000019 - city demo | rows = k | cols = RGB only + position(λ) | number = mIoU



Cityscapes classes (dark -> light = different clusters of one class)

road	fence	vegetation	rider	train
sidewalk	pole	terrain	car	motorcycle
building	traffic light	sky	truck	bicycle
wall	traffic sign	person	bus	

Wizualizacje

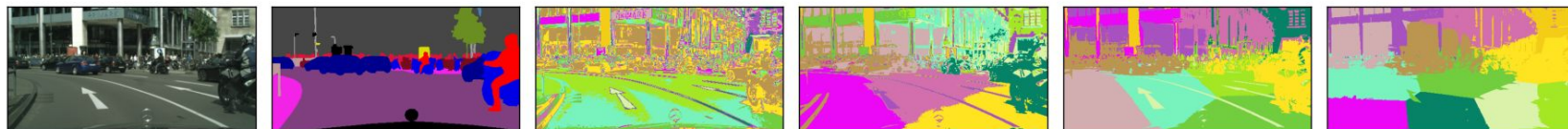


cologne_000019_000019 (k=12) - gora: kolory klas (GT) | dol: kolory losowe (bez GT)

kolory wg klas
(mapowanie GT)

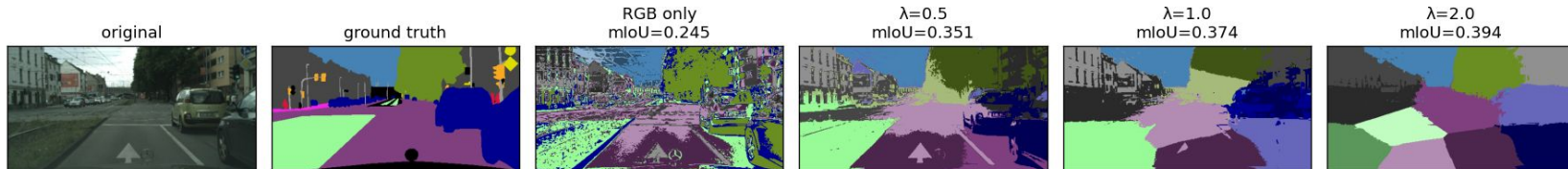


kolory losowe
(surowe klastry)

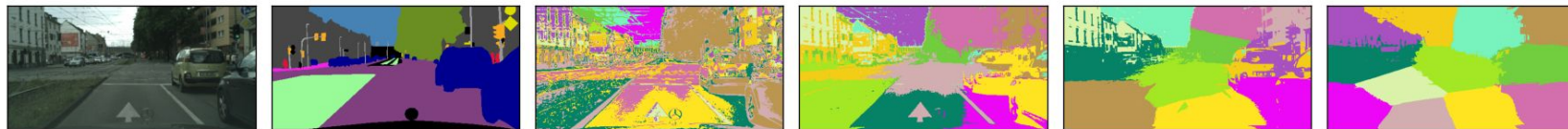


cologne_000045_000019 (k=12) - gora: kolory klas (GT) | dol: kolory losowe (bez GT)

kolory wg klas
(mapowanie GT)

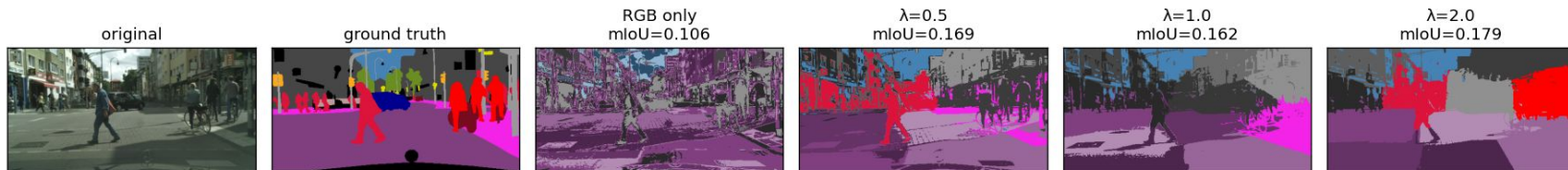


kolory losowe
(surowe klastry)

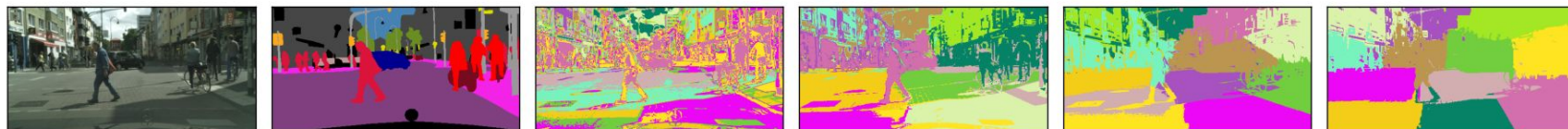


cologne_000056_000019 (k=12) - gora: kolory klas (GT) | dol: kolory losowe (bez GT)

kolory wg klas
(mapowanie GT)

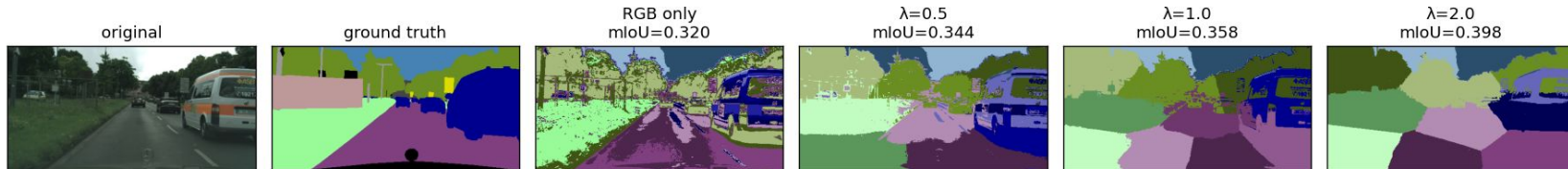


kolory losowe
(surowe klastry)

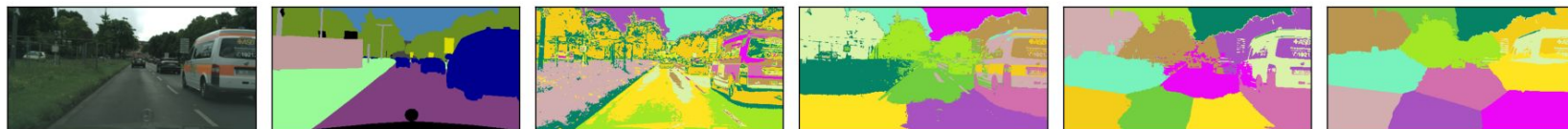


cologne_000060_000019 (k=12) - gora: kolory klas (GT) | dol: kolory losowe (bez GT)

kolory wg klas
(mapowanie GT)



kolory losowe
(surowe klastry)

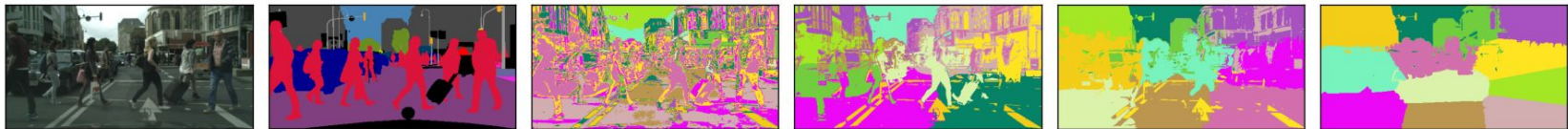


cologne_000135_000019 (k=12) - gora: kolory klas (GT) | dol: kolory losowe (bez GT)

kolory wg klas
(mapowanie GT)

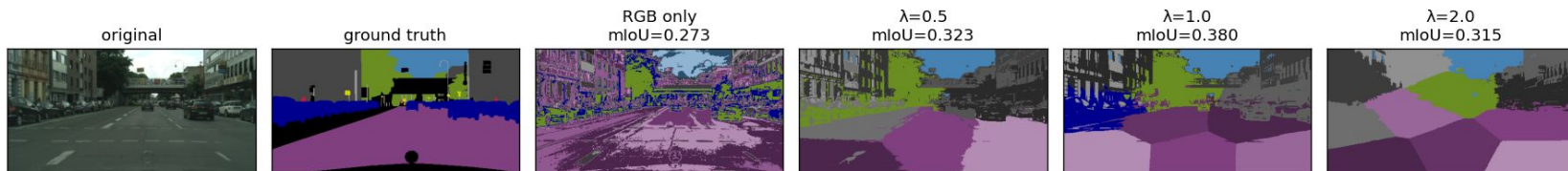


kolory losowe
(surowe klastry)

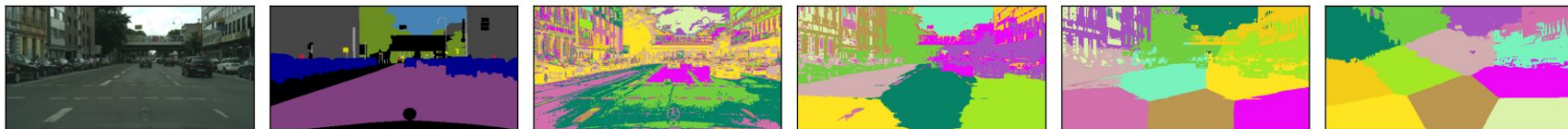


cologne_000145_000019 (k=12) - gora: kolory klas (GT) | dol: kolory losowe (bez GT)

kolory wg klas
(mapowanie GT)

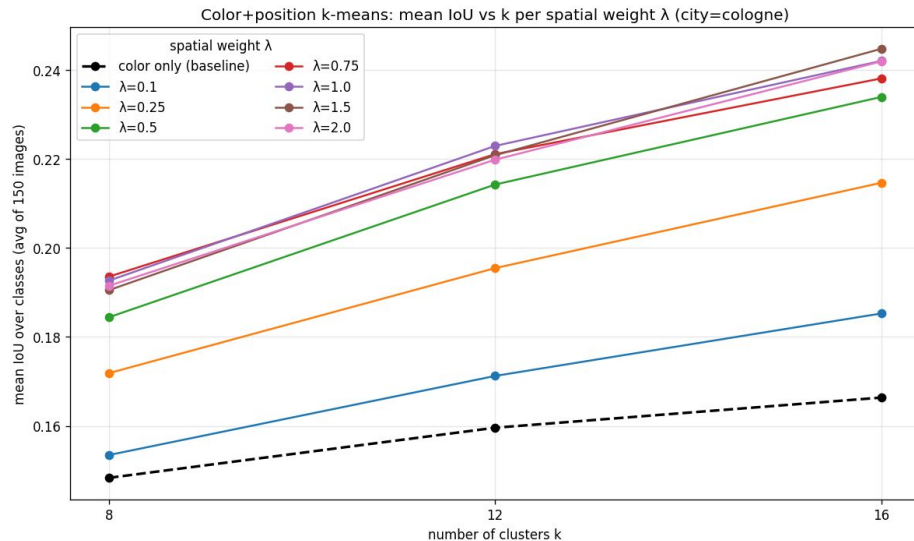


kolory losowe
(surowe klastry)



Wyniki

Większa liczba klastrów daje średnio lepszy wynik (Mniejsze obiekty nie zlewają się z większymi). Ze względu na niedokładność IoU większe wartości λ dają lepsze wyniki, dla ludzkiego oka wartość w przedziale **0.5 - 0.75** daje najlepsze rezultaty.



Wyniki

Stopień pokrycia obiektów
po przypisaniu klastrom
klas obiektów z GT

Best color+pos setting: k=16, lambda=1.5 (mIoU=0.245)

	k=16, lambda=1.5	k=12, lambda=0.5
road	0.710	0.692
sidewalk	0.146	0.095
building	0.561	0.479
wall	0.087	0.041
fence	0.033	0.009
pole	0.001	0.001
traffic light	0.000	0.000
traffic sign	0.007	0.003
vegetation	0.449	0.386
terrain	0.095	0.064
sky	0.657	0.659
person	0.029	0.014
rider	0.004	0.000
car	0.276	0.198
truck	0.017	0.018
bus	0.169	0.131
train	0.067	0.025
motorcycle	0.044	0.019
bicycle	0.030	0.011

Wnioski

Algorytm K-means wydaje się nieodpowiedni do tego zadania, bierze pod uwagę szczegóły które w GroundTruth są pomijane, metryka IoU jest niemiarodajna i nie sprawdza się w ocenie wydajności modelu.

Wizualnie najlepsze efekty daje $\lambda=0.5$, skutecznie redukuje szum, niestety czasem kosztem utraty detali. Przy dużych wartościach λ na obrazie generują się jednolite plamy, co prowadzi do uzyskania najwyższego IoU, jednak to wyłącznie efekty przypadku i pokrywania się plam z maską.

Problemy

GroundTruth traktuje budynki jako jedną, spójną całość, podczas gdy nasz algorytm nadmiernie je fragmentuje, wyodrębniając na przykład pojedyncze okna. W efekcie wynik algorytmu nie pokrywa się z danymi testowymi. Problem wynika z faktu, że nasz model skutecznie wykrywa i koloruje poszczególne detale.

Kolejnym przykładem tego zjawiska są znaki poziome na jezdni, które model wyodrębnia jako osobne obiekty. Co więcej, zachowanie modelu bywa pod tym względem skrajne – podczas gdy jedne obiekty są nadmiernie dzielone, w innych przypadkach algorytm zbytnio generalizuje obraz, zlewając ze sobą zróżnicowane elementy tła w jedną dużą klasę.

Shoutouts

@gmum

@cityscapes-dataset

<https://www.cityscapes-dataset.com/>

KONIEC

Dziękujemy za uwagę.

